

SIMATS ENGINEERING
ASSESSMENT TOOL 2: Case Study

COURSE NAME: Cloud Computing and **COURSE CODE:** CSA15
Big Data Analytics

COURSE OUTCOME COVERED

CO3: *Implement cloud-based solutions using cloud storage, web APIs, and platform services for real-world applications. (BL3)*

Dave's Taxonomy: Precision (Level 3)
Question Count: 3

Total Marks: 30

Code of Conduct

I Athavan K(192524036) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Athavan k

Assessment Tasks Case Study

1. Case Study 1: Smart Retail Inventory Management System

A retail chain implements a cloud-based inventory system connected to store devices. Clients (POS systems and dashboards) communicate via APIs to centralized cloud services. Cloud storage stores product data, transaction logs, and analytics datasets. The system uses platform services for analytics and demand forecasting. Secure access is maintained through VPNs, firewalls, and API gateways. This solution integrates infrastructure, services, and web applications for efficient operations.

2. Case Study 2: Cloud-Based Document Collaboration System

An enterprise deploys a document collaboration tool similar to Google Docs. Users access the platform via browsers, enabling real-time editing and sharing. Documents are stored in distributed cloud storage with version control mechanisms. Web APIs enable synchronization across multiple clients and devices.

Network infrastructure ensures low latency and high availability globally. Security includes encryption, identity management, and access permissions for collaboration.

3. Case Study 3: Online Food Ordering Platform

A startup builds a cloud-based food ordering system accessible via web browsers and mobile clients. Frontend clients interact with backend services through RESTful Web APIs hosted on a cloud platform.

Cloud storage is used to store menu images, invoices, and customer data securely.

The infrastructure uses virtual machines and managed Kubernetes for scalability and reliability. Security is enforced through authentication tokens, HTTPS, and role-based access control. The system demonstrates integration of clients, APIs, storage, and platform services in real time.

Case Study Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Case	25%	Clearly identifies all technical and business requirements	Identifies most requirements	Basic understanding only	Poor understanding of case
Application of Concepts	25%	Applies suitable cloud concepts and tools effectively	Good application with minor issues	Partial application	Weak or incorrect application
Analysis	20%	Strong analysis with clear trade-off reasoning	Reasonable analysis	Limited analysis	Minimal analysis
Solution Design	20%	Solution is feasible, practical, and well justified	Reasonable solution with minor gaps	Basic solution	Unclear solution
Clarity	10%	Wellstructured and easy to follow	Mostly clear	Somewhat unclear	Poorly organized

Case Study 1: Smart Retail Inventory Management System

1. Understanding of the Case

The retail chain requires a cloud-based inventory management system that connects hundreds of physical stores to a centralized platform. The core technical requirements are: (a) real-time synchronisation between Point-of-Sale (POS) devices, dashboards, and the cloud backend; (b) centralised, durable storage for product catalogues, transaction logs, and analytics datasets; (c) platform services capable of running demand-forecasting and analytics workloads; and (d) enterprise-grade security spanning network access (VPN, firewalls) and API-level access control (API gateways). The business requirements are equally clear: minimise stock-outs and overstocking through accurate forecasting, give store managers and head-office a single source of truth for inventory, and keep the system available and secure across a geographically distributed store network.

2. Application of Cloud Concepts and Tools

- **Clients:** POS terminals and manager/head-office dashboards act as thin clients that call backend services only through defined APIs, decoupling store hardware from backend logic.
- **Web APIs:** A RESTful API layer (fronted by an API Gateway such as AWS API Gateway / Azure API Management) exposes endpoints for inventory updates, sales transactions, and analytics queries, with versioning to support store hardware upgrades independently of the backend.
- **Cloud Storage:** Product data and transaction logs are stored in a managed relational database (e.g., Amazon RDS/Azure SQL) for transactional consistency, while analytics datasets and historical logs are stored in low-cost object storage (e.g., Amazon S3/Azure Blob) and a data lake for large-scale processing.
- **Platform Services (PaaS):** A managed analytics/ML service (e.g., Amazon Forecast, Azure Machine Learning) is used for demand forecasting so the retailer does not need to manage forecasting infrastructure, and a serverless compute layer (e.g., AWS Lambda/Azure Functions) processes incoming POS events.
- **Security:** VPN tunnels connect store networks to the cloud VPC, firewalls and security groups restrict traffic to required ports/services, the API gateway enforces authentication (OAuth2/API keys) and rate limiting, and data is encrypted in transit (TLS) and at rest.

3. Analysis (Trade-offs)

A key design decision is choosing a relational database for transactional inventory data versus a NoSQL store. RDS/SQL gives strong consistency, which is essential so that two stores never oversell the same stock unit, but it scales vertically and can become a bottleneck under very high write volume. A hybrid approach mitigates this: transactional writes go to a scalable managed SQL service with read replicas, while historical and analytical data is offloaded to a data lake, trading some query flexibility on the hot path for strong consistency and lower cost on the analytics path.

A second trade-off is centralisation versus store-level autonomy. Fully centralising all logic in the cloud simplifies management and forecasting accuracy but makes stores dependent on connectivity; a brief internet outage could halt sales. The recommended mitigation is edge caching at the POS level (local queueing of transactions that sync once connectivity resumes), trading a small amount of architectural complexity for high availability at the store.

Security-wise, a VPN-only model is simpler to configure but scales poorly across hundreds of stores and creates a single large trust boundary; an API-gateway-centric zero-trust model (every request authenticated regardless of network origin) is more resilient to a single compromised store but requires more sophisticated identity management. The zero-trust approach is recommended given the scale of the retail chain.

4. Solution Design

The proposed architecture uses store POS/dashboard clients communicating over an encrypted VPN link to a cloud VPC. All requests pass through an API Gateway that authenticates via OAuth2 tokens and forwards to a serverless application layer. Transactional data (stock levels, sales) is written to a managed relational database with automated backups and read replicas for dashboard queries; product images and bulk logs go to object storage. A scheduled ETL pipeline moves transaction data into a data lake, where a managed ML forecasting service generates demand predictions consumed by a replenishment dashboard. Firewalls/security groups restrict database access to the application tier only, and all data is encrypted at rest and in transit.

Case Study 2: Cloud-Based Document Collaboration System

1. Understanding of the Case

The enterprise needs a Google-Docs-like collaboration platform supporting concurrent, real-time editing by multiple users across the globe. Technical requirements include browser-based clients with no local installation, distributed cloud storage with version history, synchronisation APIs that propagate edits across all connected clients with minimal latency, and a global network architecture that keeps latency low regardless of user location. Business requirements include strong security (encryption, identity management, granular access permissions) since documents may contain sensitive corporate information, and high availability so that collaboration is never blocked by a regional outage.

2. Application of Cloud Concepts and Tools

- **Clients:** Modern browsers act as the client, using WebSockets or a similar persistent-connection protocol for real-time updates rather than simple request/response polling.
- **Web APIs:** RESTful APIs handle document CRUD operations (create, open, share, list), while a WebSocket-based real-time API synchronises live edits between clients using an Operational Transformation (OT) or Conflict-free Replicated Data Type (CRDT) algorithm to merge concurrent changes.
- **Cloud Storage:** Documents and their version history are stored in distributed object storage (e.g., Amazon S3/Azure Blob) with versioning enabled, while document metadata (owners, permissions, last-edited timestamps) is kept in a fast NoSQL database (e.g., DynamoDB/Cosmos DB) for low-latency lookups.
- **Platform Services and Network:** A Content Delivery Network (CDN) and multi-region deployment reduce latency for globally distributed users, and managed load balancers route traffic to the nearest healthy region.
- **Security:** Identity and Access Management (IAM) integrated with Single Sign-On (SSO) authenticates users; role-based access control governs document-level permissions (view/comment/edit/owner); and TLS in transit plus AES-256 at rest protect document content.

3. Analysis (Trade-offs)

The central technical trade-off is the real-time synchronisation algorithm. Operational Transformation is battle-tested (used by Google Docs) and handles rich text well, but it is complex to implement correctly and requires a central server to sequence operations. CRDTs simplify conflict resolution and work well in peer-to-peer or offline-first scenarios, but can have higher memory overhead for large documents. Given the enterprise context (centralised control and auditability are valued), a server-mediated OT approach is recommended, accepting implementation complexity in exchange for simpler conflict guarantees and centralised audit logging.

A second trade-off concerns storage consistency versus availability. Storing every document version (full version history) supports rollback and compliance but increases storage cost and cross-region replication time. Multi-region eventual consistency improves availability and latency for global users but risks a user briefly seeing a stale version after a region failover. The recommended balance is strong consistency within a region (so active collaborators always see the same state) combined with asynchronous, eventually consistent replication across regions for disaster recovery.

Finally, broad RBAC permissions are simple to manage but risk over-provisioning access; fine-grained, document-level ACLs are more secure but add administrative overhead. Given that the platform handles potentially sensitive enterprise documents, fine-grained ACLs enforced at the API layer are justified despite the extra management cost.

4. Solution Design

Users authenticate through enterprise SSO/IAM and connect via browser clients over TLS. A REST API handles document lifecycle operations while a WebSocket gateway, backed by an OT engine, streams and merges live edits between all active sessions on a document. Documents and revision history are stored in versioned object storage; metadata and permissions live in a globally distributed NoSQL store replicated across regions. A CDN caches static assets and read-heavy content close to users, and a global load balancer routes each user to their nearest region, with asynchronous cross-region replication providing disaster recovery. RBAC combined with document-level ACLs, enforced at the API gateway, ensures only authorised users can view, comment on, or edit a given document. This design meets the low-latency, high-availability, and security requirements while remaining implementable with mainstream managed cloud services.

Case Study 3: Online Food Ordering Platform

1. Understanding of the Case

The startup needs a scalable food ordering platform accessible via web and mobile clients, with a backend exposed through RESTful APIs. Technical requirements include storage for menu images, invoices, and customer data; a scalable compute layer capable of handling variable and often spiky traffic (e.g., meal-time peaks); and strong security given that the platform handles personal and payment-related customer data. Business requirements include reliability during high-demand periods, the ability to scale cost-effectively as a startup grows, and compliance-friendly handling of customer data.

2. Application of Cloud Concepts and Tools

- **Clients:** Web and mobile apps (Android/iOS) act as frontend clients that communicate exclusively through RESTful Web APIs, allowing the same backend to serve multiple client types.
- **Web APIs:** REST APIs expose endpoints for menu browsing, order placement, order tracking, and payment initiation, secured using authentication tokens (JWT/OAuth2) so only verified clients and users can act on an account.
- **Cloud Storage:** Menu images are stored in object storage (e.g., Amazon S3/Azure Blob) served through a CDN for fast image delivery; invoices and customer transactional data are stored in a managed relational database for consistency and easy reporting/auditing.
- **Platform Services:** Virtual machines handle steady baseline workloads, while managed Kubernetes (e.g., Amazon EKS/Azure AKS) auto-scales stateless services such as the order-processing and notification microservices during demand spikes, giving elasticity without manual server provisioning.
- **Security:** HTTPS/TLS encrypts all client-server traffic, authentication tokens (JWT) validate every API request, and role-based access control differentiates customers, restaurant partners, and delivery/admin staff.

3. Analysis (Trade-offs)

A central trade-off is the choice between virtual machines and managed Kubernetes for different parts of the workload. VMs are simpler to reason about and suit predictable, always-on components (e.g., the core database), but they scale slowly and can be cost-inefficient for bursty traffic. Kubernetes-based containers enable rapid horizontal auto-scaling for the API and order-processing layers during lunch/dinner rushes, at the cost of added operational complexity (cluster management, container orchestration expertise). The recommended approach uses VMs/managed database services for stateful, steady components and Kubernetes for stateless, bursty services -- balancing simplicity where load is predictable against elasticity where it is not.

A second trade-off is storing customer and payment-related data directly versus using a third-party payment gateway. Directly storing card data would minimise dependency on external providers but massively increases PCI-DSS compliance burden and security risk, which is disproportionate for a startup. Delegating payment capture to a certified gateway (e.g., Stripe/Razorpay) and storing only tokenised references trades some control for significantly reduced compliance scope and risk -- the correct choice for a startup with limited security resources.

Finally, using a single-region deployment keeps costs low and architecture simple, which suits an early-stage startup, but risks downtime if that region fails. Multi-region deployment improves resilience but multiplies infrastructure cost. Given the startup context, a single-region deployment with automated backups and a documented disaster-recovery plan is a pragmatic first step, with multi-region expansion planned as the business scales.

4. Solution Design

Web and mobile clients call a REST API layer secured with JWT-based authentication and served over HTTPS. Stateless services (order placement, order tracking, notifications) run as containers on a managed Kubernetes cluster that auto-scales based on CPU/request-rate metrics during peak hours, while the core customer/order/invoice database runs on a managed relational database service with automated backups. Menu images are stored in object storage and delivered via a CDN for fast load times on mobile networks. Payments are processed through a certified third-party payment gateway, with only a tokenised reference stored in the platform's database, keeping PCI-DSS scope minimal. Role-based

access control differentiates customer, restaurant-partner, and admin permissions at the API layer. This design is practical for a startup's budget, scales elastically during demand spikes, and keeps sensitive payment handling outside the platform's direct responsibility.

Overall Summary

Across all three case studies, the common cloud architecture pattern is a client layer (web/mobile/POS) communicating exclusively through secured Web APIs to backend platform services, with data persisted in a mix of cloud object storage (for unstructured/bulk content) and managed database services (for structured, consistent transactional data). Security is layered at the network level (VPN/firewalls), the API level (gateways, tokens, TLS), and the identity level (IAM/RBAC). The key engineering judgement in each case is matching the right storage and compute service to each workload's consistency, latency, and scale requirements, rather than applying a single uniform solution across all components.